Analysis

Duncan Parke

Data Structures

This lab uses a priority queue (min-heap) to build the Huffman tree, an unordered_map to store the frequency table, and another unordered_map to store the Huffman codes. The priority queue was chosen because it efficiently supports the repeated extraction of the smallest element, which is essential for constructing the Huffman tree. The unordered map was used for its average O(1) time complexity for lookups and insertions, making it ideal for storing and accessing character frequencies and codes.

Design

The program reads a frequency table and uses it to build a Huffman tree. The tree is constructed by repeatedly merging the two nodes with the smallest frequencies until a single tree remains. This process ensures that the most frequent characters have the shortest codes, achieving data compression.

The design includes functions to:

- recursively build the huffman tree
- generate huffman codes by traversing the tree
- encode a given text using the generated codes
- decode an encoded text back to its original form.
- print the tree

The program also preprocesses the input text to match the frequency table, ensuring consistency in encoding and decoding.

Efficiency

The time complexity of building the Huffman tree is O(nln), where n is the number of unique characters. Generating the codes involves a tree traversal, which is O(n). Encoding and decoding are linear with respect to the length of the text. The space complexity is O(n) for storing the maps & tree, so the program is relatively efficient in both time and space

What I Learned

This lab reinforced the importance of choosing the right data storage for specific tasks. The priority queue was crucial for efficiently building the huffman tree, and the unorderd map simplified frequency counting. I also learned that its hard to know where to put spaces when you don't have it in the frequency list. I also learned some of thje background on lorim ipsum, since i was looking for common phrases/texts that might be interesting to compress.

What I Might Do Differently

In the future, I might explore alternative tie-breaking strategies when merging nodes in the priority queue, such as prioritizing alphabetical order. This could make the tree structure more predictable. I would also

consider implementing more error handling to "idiot proof" the code a bit, since I feel like people would want to try to set this on all sorts of things to compress. I would also want some mechanism for maybe trying to guess the ends of words and insert spaces, but that would be much more complex than just encoding spaces. I might also do more to handle the situation in which we get a character not included in the freq tree

Enhancements

The program includes several enhancements:

- The huffman tree is visualized using something similar to the tree command in linux, but also incorperates the ones and zeros that would generate the table
- It preprocesses the input text to ensure compatibility with the frequency table, which is less interesting given a freq tree with a full alphabet
- The program reports compression ratio

Compression Analysis

The program achieves data compression by assigning shorter codes to more frequent characters. For example, the encoded text is significantly smaller than the original clear text, with the given example coming in at 54% the size of the original text, which is pretty good.

If I used a different mechanism to break ties, compression results would vary by different pieces of text as to whether or not the character which recieved precedence was more or less common in the text. This would generally be a negative impact I believe, especailly if the frequency table is not true to the underlying data. but if it is, I think we'd end up pretty much at parity